Files

main

Go to file

- ML0.1 intro
- ML0.2 simple analisys
- ML1.1 linear regression
- ML1.2 multivariate
- ML1.3 real regression
- ML2.1 logistic regression
- ML2.2 real classification
- ML3.1 polynomial features
- ML3.2 svm
- ML3.3 mlp
- ML3.4 trees
- ML3.5 knn
- ML3.6 bayes
- ML4.1 metrics
- ML4.2 threshold
- ML4.3 cross-validation
- ML4.4 diagnostics
- ML4.5 hyperparameters
- ML5.1 data integration

| Name | Last commit message | Last commit date |
|---|---|---|
| .. | | |
| data | Merge branch '2023' | 3 years ago |
| img | Картинки распределены по папкам | 2 years ago |
| README.md | Исправлены ссылки на картинки | last year |

README.md

# Модели классификации

## Цель работы

Познакомиться с основными приемами работы с моделями классификации в scikit-learn.

## Задания для выполнения

1. Загрузите данные о диагностике сахарного диабета.
2. Постройте модель классификации для предсказания наличия заболевания.
3. Оцените качество построенной модели с помощью отчета о классификации и матрицы классификации.
4. Постройте альтернативную полиномиальную модель, сравните ее с предыдущей.

## Методические указания

Для начала работы обратимся к набору данных diabetes. Это довольно известный датасет, собравший информацию о медицинских показателях более 700 пациентов, обследованных на предмет наличия сахарного диабета. На нем мы потренируемся строить классификационные модели.

Сперва загрузим исходный набор данных. Это можно сделать, как скопировав файл csv в локальную папку, так и по общедоступному URL:

```
data = pd.read_csv("https://raw.githubusercontent.com/koroteevmv/ML_course/2023/ML2.2%20real%20classification/data/diabetes.csv'
```

Обратите внимание, что в библиотеке *sklearn* встроен очень похожий датасет pima-indian-diabetes. Имейте в виду, что в данной работе используется немного другой датасет.

Как и ранее, хорошей идеей перед началом анализа будет познакомиться с составом набора данных визуально. Выведем датасет на экран:

```
data.head()
```

Познакомьтесь с основной структурой датасета:

| index | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI | DiabetesPedigreeFunction | Age | Outcome |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 6 | 148 | 72 | 35 | 0 | 33.6 | 0.627 | 50 | 1 |
| 1 | 1 | 85 | 66 | 29 | 0 | 26.6 | 0.351 | 31 | 0 |
| 2 | 8 | 183 | 64 | 0 | 0 | 23.3 | 0.672 | 32 | 1 |
| 3 | 1 | 89 | 66 | 23 | 94 | 28.1 | 0.167 | 21 | 0 |
| 4 | 0 | 137 | 40 | 35 | 168 | 43.1 | 2.288 | 33 | 1 |

При проведении серьезного анализа перед построением модели машинного обучения нужно провести тщательную обработку и очистку набора данных - удаление пропущенных значений, анализ шкал, нормализация, удаление выбросов и аномалий. В учебных целях ограничимся обязательными проверками критических ошибок в данных.

В первую очередь проверим данные на наличие пропущенных значений:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

Видим, что пропусков в данных нет. Кроме того, видно, что все данные выражены в численных шкалах. Значит, особенной обработки данный датасет не требует, он уже достаточно чистый. Теперь можно вывести основную статистику по датасету:

```
data.describe()
```

| index | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | |
|---|---|---|---|---|---|---|
| count | 768.0 | 768.0 | 768.0 | 768.0 | 768.0 | 768.0 |
| mean | 3.8450520833333335 | 120.89453125 | 69.10546875 | 20.536458333333332 | 79.79947916666667 | 31.9925 |
| std | 3.3695780626988694 | 31.97261819513622 | 19.355807170644777 | 15.952217567727637 | 115.24400235133817 | 7.88416 |
| min | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| 25% | 1.0 | 99.0 | 62.0 | 0.0 | 0.0 | 27.3 |
| 50% | 3.0 | 117.0 | 72.0 | 23.0 | 30.5 | 32.0 |
| 75% | 6.0 | 140.25 | 80.0 | 32.0 | 127.25 | 36.6 |
| max | 17.0 | 199.0 | 122.0 | 99.0 | 846.0 | 67.1 |

Здесь мы видим шкалу измерения каждого признака. Можно придти к выводу, что явных видимых аномалий в данных нет. отдельно обратим внимание на столбец "Outcome" - он содержит целевую переменную. В данном случае она также выражается числом (0 - здоров, 1 - болен), но это не всегда так.

Теперь выделим целевую переменную и факторы:

```
y = data.Outcome
X = data.drop(["Outcome"], axis=1)
```

Выведем форму получившихся массивов:

```
y.shape, X.shape
```

```
((442,), (442, 10))
```

Данные выглядят полностью готовыми к началу машинного обучения. Для начала импортируем нужный класс и создадим его экземпляр:

```
from sklearn.linear_model import LogisticRegression
logistic = LogisticRegression()
```

Обучим созданную модель на имеющихся у нас данных:

```
logistic.fit(X, y)
```

После выполнения этой инструкции мы можем увидеть специальное предупреждение:

```
/usr/local/lib/python3.8/dist-packages/sklearn/linear_model/_logistic.py:814: ConvergenceWarning: lbfgs failed to converge (stat
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
LogisticRegression()
```

Смысл этого сообщения в том, что процесс обучения завершился по условию достижения максимального количества итераций, а не по условию стабилизации функции ошибки. Это значит, что модель обучается трудно и медленно. Это может свидетельствовать о том, что результаты могут быть не очень удовлетворительными.

Но давайте посмотрим, что за модель мы получили после такого обучения. В первую очередь выведем коэффициенты модели:

```
print("Coefficients: \n", logistic.coef_[0])
```

```
Coefficients: 
 [ 1.17252338e-01  3.35998450e-02 -1.40873757e-02 -1.27047756e-03
 -1.24032162e-03  7.72023360e-02  1.41904174e+00  1.00353608e-02]
```

В линейных моделях коэффициенты имеют физический смысл - они показывают значимость соответствующих признаков. Поэтому представляет особый интерес посмотреть коэффициенты вместе с названиями признаков.

Для этого соединим массив названий колонок из датасета и массив коэффициентов. Можно использовать, например, генераторное выражение для прохода по получившемуся массиву. Конструкция "_ = [ ... ]" нужна только в ноутбуке для того, чтобы подавить автоматический вывод выражения:

```
_ = [print(k, v) for k, v in zip(X.columns, logistic.coef_[0])]
```

```
Pregnancies 0.11725233809260834
Glucose 0.03359984495281752
BloodPressure -0.014087375706652597
SkinThickness -0.0012704775628293837
Insulin -0.001240321623795555
BMI 0.07720233600112382
DiabetesPedigreeFunction 1.4190417369640222
Age 0.010035360773350831
```

Самостоятельно проанализируйте эти данные и сделайте вывод, какие атрибуты оказывают большее влияние на значение целевой переменной.

Как и в модели линейной регрессии, данный вектор не включает в себя свободный коэффициент. Он хранится в отдельном поле класса:

```
print("Intercept: \n", logistic.intercept_)
```

```
Intercept: 
 [-7.70291388]
```

Теперь можно построить по полученной модели прогноз. Для этого передадим в соответствующий метод нашу матрицу признаков:

```
y_pred = logistic.predict(X)
```

Сформировав вектор предсказанных значений целевой переменной можно сравнить его с реальными значениями:

```
_ = [print(a, b) for a, b in list(zip(y, y_pred))[:10]]
```

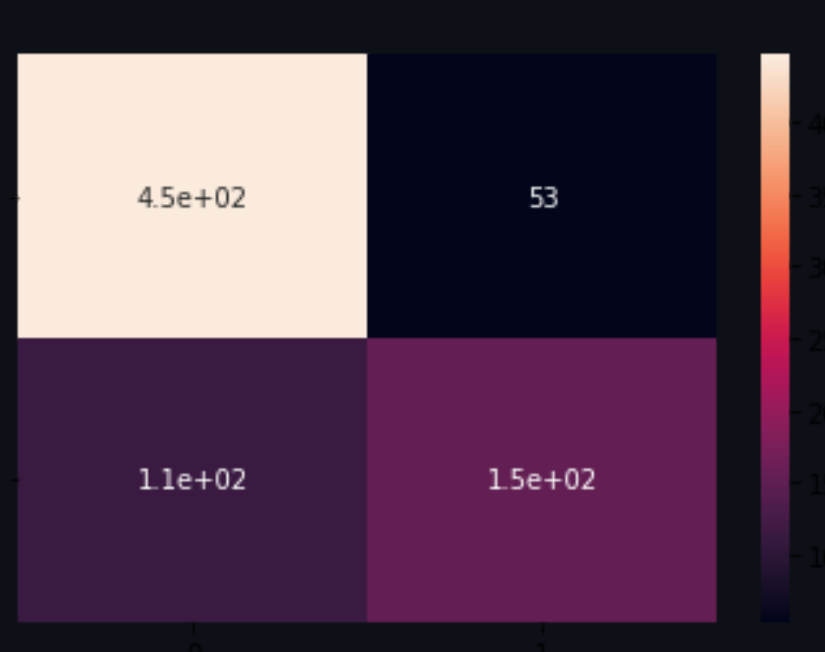
Можно видеть, что большинство значений совпадает, но есть и ошибки - различия в значениях. Но так сравнивать все значения в ручном режиме очень неудобно. Поэтому лучше использовать специальные функции - метрики. Самая простая из них подсчитывает количество правильно и неправильно распознанных объектов и представляет результат в виде матрицы классификации:

```
from sklearn import metrics
metrics.confusion_matrix(y, y_pred)
```

Матрица классификации показывает нам очень полезную информацию: совместное распределение числа объектов предсказанных и реальных классов. Рассматривая эту матрицу мы можем получить важную информацию: сколько объектов мы классифицировали правильно, сколько неверно, к каким классам наша модель тяготеет, какие классы распознаются хорошо, какие - плохо

Гораздо удобнее анализировать ту же информацию в графической форме. Для этого воспользуемся специальной библиотекой *seaborn*, которая позволяет создавать полезные для машинного обучения визуализации очень просто:

```
import seaborn as sns
sns.heatmap(metrics.confusion_matrix(y, y_pred), annot=True)
```



Кроме матрицы классификации весьма полезно использовать численные метрики эффективности классификации. Самая простая и распространенная из них - метрика точности предсказания - показывает долю правильно распознанных объектов. Расчет этой метрики встроен в сам объект модели и доступен с помощью специального метода:

```
logistic.score(X, y)
```

```
0.7825520833333334
```

Эту же метрику можно рассчитать и по-другому - через отдельную функцию из пакета *metrics*. Обратите внимание на другую сигнатуру метода:

```
metrics.accuracy_score(y_test, y_pred)
```

Значение метрики (0,78) показывает, что модель в среднем делает ошибки в 22% процентов случаев. Это основной показатель качества модели. В дальнейших работах мы покажем, как его замерять более правильно.

Если же такой уровень эффективности нас не устраивает, то мы можем попробовать использовать другие классы моделей классификации и среди них выбрать наиболее качественную. Например, можно попробовать построить полиномиальную модель. В библиотеке *sklearn* не предусмотрено отдельного класса полиномиальной модели. Ее можно создать через специальный объект *PolynomialFeature*, который добавляет полиномиальные признаки к данным. Для его использования сначала импортируем его:

```
from sklearn.preprocessing import PolynomialFeatures
```

Теперь можно создать объект преобразования (точно также как мы создавали объект модели):

```
poly = PolynomialFeatures(2)
```

Здесь мы указываем, что будем создавать полиномиальные признаки второго порядка. Теперь можно использовать этот объект для создания собственно самих признаков:

```
poly = poly.fit_transform(X)
poly
```

Этот код выводит следующие данные на экран:

```
array([[1.00000e+00, 6.00000e+00, 1.48000e+02, 7.20000e+01, 3.50000e+01,
        0.00000e+00, 3.36000e+01, 6.27000e-01, 5.00000e+01, 3.60000e+01,
        8.88000e+02, 4.32000e+02, 2.10000e+02, 0.00000e+00, 2.01600e+02,
        3.76200e+00, 3.00000e+02, 2.19040e+04, 1.06560e+04, 5.18000e+03,
        0.00000e+00, 4.97280e+03, 9.27960e+01, 7.40000e+03, 5.18400e+03,
        2.52000e+03, 0.00000e+00, 2.41920e+03, 4.51440e+01, 3.60000e+03,
        1.22500e+03, 0.00000e+00, 1.17600e+03, 2.19450e+01, 1.75000e+03,
        0.00000e+00, 0.00000e+00, 0.00000e+00, 0.00000e+00, 1.12896e+03,
        2.10672e+01, 1.68000e+03, 3.93129e-01, 3.13500e+01, 2.50000e+03],
       [1.00000e+00, 1.00000e+00, 8.50000e+01, 6.60000e+01, 2.90000e+01,
        0.00000e+00, 2.66000e+01, 3.51000e-01, 3.10000e+01, 1.00000e+00,
        8.50000e+01, 6.60000e+01, 2.90000e+01, 0.00000e+00, 2.66000e+01,
        3.51000e-01, 3.10000e+01, 7.22500e+03, 5.61000e+03, 2.46500e+03,
        0.00000e+00, 2.26100e+03, 2.98350e+01, 2.63500e+03, 4.35600e+03,
        1.91400e+03, 0.00000e+00, 1.75560e+03, 2.31660e+01, 2.04600e+03,
        8.41000e+02, 0.00000e+00, 7.71400e+02, 1.01790e+01, 8.99000e+02,
        0.00000e+00, 0.00000e+00, 0.00000e+00, 0.00000e+00, 7.07560e+02,
        9.33660e+00, 8.24600e+02, 1.23201e-01, 1.08810e+01, 9.61000e+02]])
```

Теперь эти данные можно использовать как исходные для моделирования. А строить мы будем обычную логистическую регрессию:

```
polynomial = LogisticRegression()
polynomial.fit(poly, y)
y_pred_poly = polynomial.predict(poly)
```

Исследуйте точность этой модели и сравните ее с линейной самостоятельно.

## Контрольные вопросы

1. Чем отличается применение разных моделей классификации в бибилиотеке *sklearn*?
2. Что показывает метрика точности регрессии?
3. Какое значение имеют коэффициенты логистической регрессии?
4. Что показывает матрица классификации?
5. Какие параметры имеет конструктор объекта логистической регрессии?
6. Какие атрибуты имеет объект логистической регрессии?
7. Какие параметры и атрибуты имеют объекты других моделей машинного обучения библиотеки *sklearn*?

## Задания для самостоятельного выполнения

1. Изучите документацию *sklearn*, посвященную классу LogisticRegression. Какую еще информацию можно вывести для обученной модели? Попробуйте изменить аргументы при создании модели и посмотрите, как это влияет на качество предсказания.
2. Попробуйте применить к той же задаче другие модели классификации. Для каждой из них выведите матрицу классификации и оценку точности. Рекомендуется исследовать следующие модели:
   i. Метод опорных векторов
      a. Без ядра
      b. С линейным ядром
      c. С гауссовым ядром
      d. С полиномиальным ядром
   ii. Метод ближайших соседей
   iii. Многослойный перцептрон
   iv. Дерево решений
   v. Наивный байесовский классификатор
   vi. (*) Другие методы:
      a. Пассивно-агрессивный классификатор
      b. Гребневый классификатор
      c. Случайный лес
      d. Беггинг
      e. Другие модели по желанию
3. Напишите функцию, которая автоматически обучает все перечисленные модели и для каждой выдает оценку точности.
4. Повторите полностью анализ для другой задачи - распознавание вида ириса по параметрам растения (можно использовать метод sklearn.datasets.load_iris()).